LIBRARY MANAGEMENT SYSTEM

A lightweight desktop application for book cataloging, search, and circulation tracking built with Python, Tkinter, and SQLite.

Language: Python 3 **GUI: Tkinter** **Database: SQLite3** **License: MIT**

PROJECT OVERVIEW

The **Mini Library Management System** provides an intuitive graphical interface for small-to-medium libraries or personal book collections. Designed with simplicity in mind, it allows users to keep track of stored inventory, check availability, issue books to members, and process returns in real time.

KEY FEATURES

Book Cataloging

Effortlessly add new books with titles and author details directly into the persistent SQLite database.

Real-Time Search

Filter through book records instantly by searching matching titles or author names.

Circulation Management

Issue books or return checked-out inventory with built-in validation preventing double-issuing.

Record Deletion

Safely delete legacy or lost entries with integrated user confirmation dialogs.

DATABASE SCHEMA

The application automatically initializes a single SQLite table named `books` upon launch if it does not already exist:

COLUMN NAME	DATA TYPE	CONSTRAINTS	DESCRIPTION
`id`	INTEGER	PRIMARY KEY, AUTOINCREMENT	Unique numerical ID for each book.
`title`	TEXT	NOT NULL	Full title of the book.
`author`	TEXT	NOT NULL	Author of the book.
`status`	TEXT	DEFAULT 'Available'	Current state (`Available` or `Issued`).

> **Database Storage Note**
> The SQLite database file `library.db` is created automatically in the working directory during the first application run. No prior database installation or server setup is required.

PREREQUISITES & SETUP

- **Python 3.x** installed on your machine.
- Standard library dependencies: `tkinter` and `sqlite3` (included by default in standard Python installations).

INSTALLATION & EXECUTION

To run the Library Management System on your local machine, follow these steps:

1. Save the source code file as `Library_management_system.py`.
2. Open your command terminal or standard command prompt.
3. Navigate to the folder containing the file:

```
cd /path/to/your/project
```

4. Execute the Python script:

```
python Library_management_system.py
```

INTERFACE & USER GUIDE

ACTION	PROCEDURE
Adding a Book	Enter the title and author in the top entry fields and click **Add Book**.
Searching Books	Type a keyword into the search bar and click **Search**. Click **Show All** to reset the view.
Issuing a Book	Select an available item from the listbox and click **Issue Book**. Status updates to 'Issued'.
Returning a Book	Select an issued item from the listbox and click **Return Book**. Status reverts to 'Available'.
Deleting a Book	Select a record, click **Delete Book**, and confirm the prompt to permanently erase the entry.

CODE ARCHITECTURE OVERVIEW

The code is structured into modular function blocks managing database transactions and user interface interactions:

```
Library_management_system.py
├── Database Setup       # sqlite3 connection & table creation
├── Functions
│   ├── refresh_books()  # Fetches and renders records into Listbox
│   ├── add_book()       # Inserts new records into DB
│   ├── search_book()    # Handles SQL LIKE queries
│   ├── issue_book()     # Validates & updates status to 'Issued'
│   ├── return_book()    # Validates & updates status to 'Available'
│   └── delete_book()    # Prompt confirmation & DELETE execution
└── GUI Layout           # Tkinter Root, Frames, Widgets, and Event Loop
```

> **Troubleshooting GUI Errors**
> If running on Linux and receiving a Tkinter missing error, install it using your system package manager (e.g., `sudo apt install python3-tk`).